

bioinformatics.ca

bioinformaticsdotca.github.io

Classification using Artificial Neural Networks

Zahra Sadeghi
Zahra.ai@pm.me

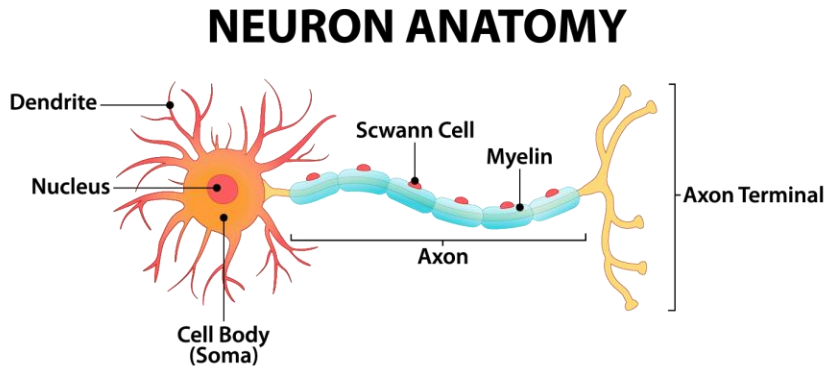
14/10/2026-15/10/2026

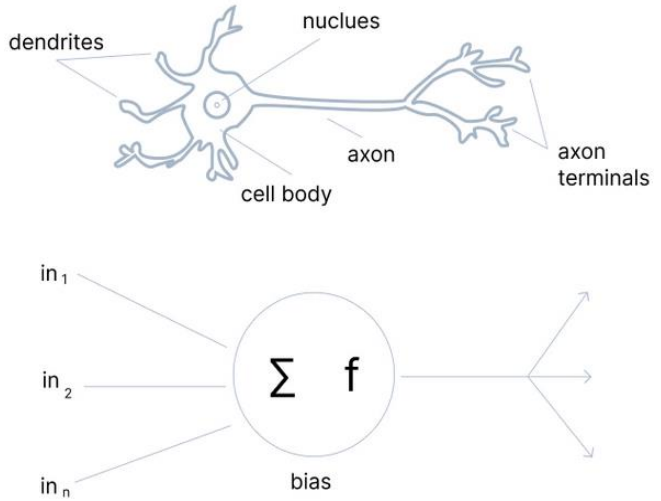


Neuron and Perceptron

Artificial Neuron

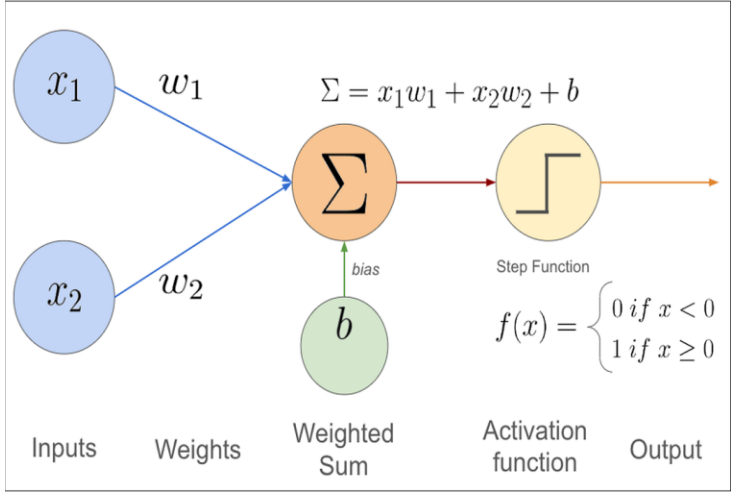
- An artificial neuron takes several inputs, assigns **weights** to them, adds a **bias**, and applies an **activation function**:

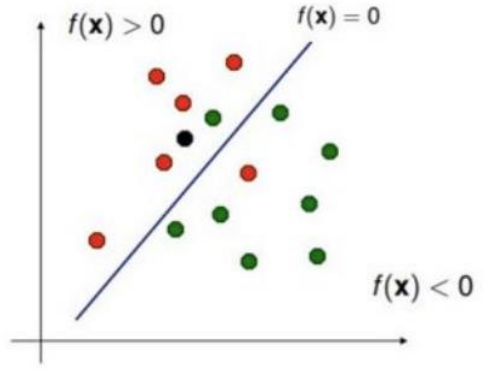




Perceptron

- A **perceptron** is one of the simplest artificial neurons.
- **Inputs xi** — information provided to the neuron
- **Weights wi** — importance assigned to each input
  - Controls how strongly input xi influences the neuron.
  - Large positive weight → strong positive influence
  - Large negative weight → strong negative influence
  - Weight near zero → weak influence
- **Bias b** — shifts the decision boundary
- For binary classification:
  - The perceptron therefore learns a **linear decision boundary**.
  - **A perceptron learns how to combine input features to make a simple decision.**





$$z = w^T x + b$$

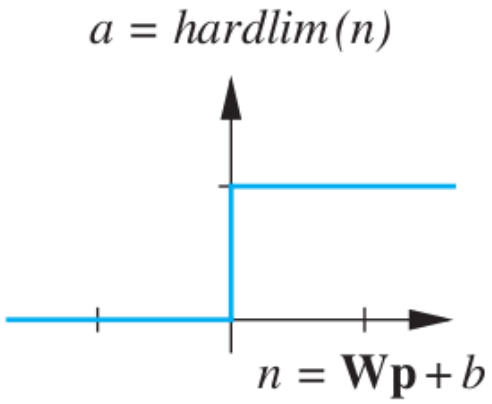
$$\hat{y} = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

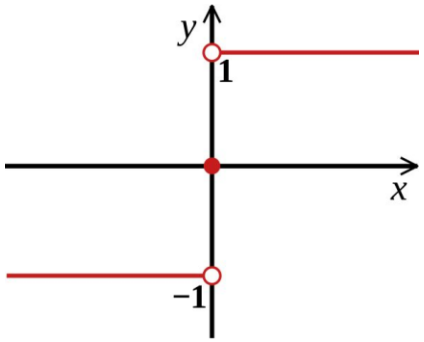
$$z \rightarrow \text{hard threshold} \rightarrow \{0, 1\}$$

The value z is called a **logit**.

Perceptron

$$\mathbf{a} = \mathbf{hardlim}(\mathbf{Wp} + \mathbf{b}) .$$





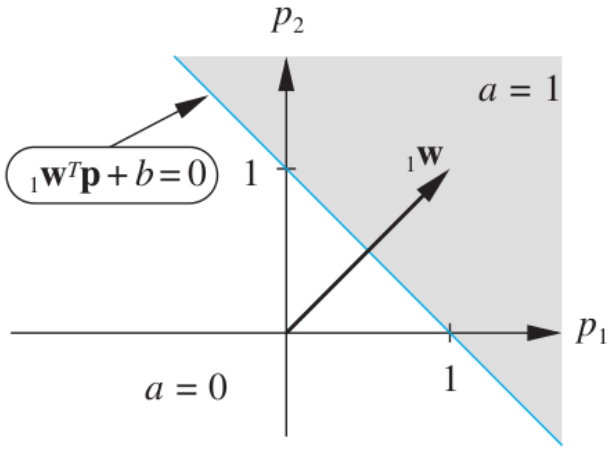
Perceptron learning rule: how do we update the weight vector?

$$e = y - \hat{y}$$

$$w \leftarrow w + ex$$

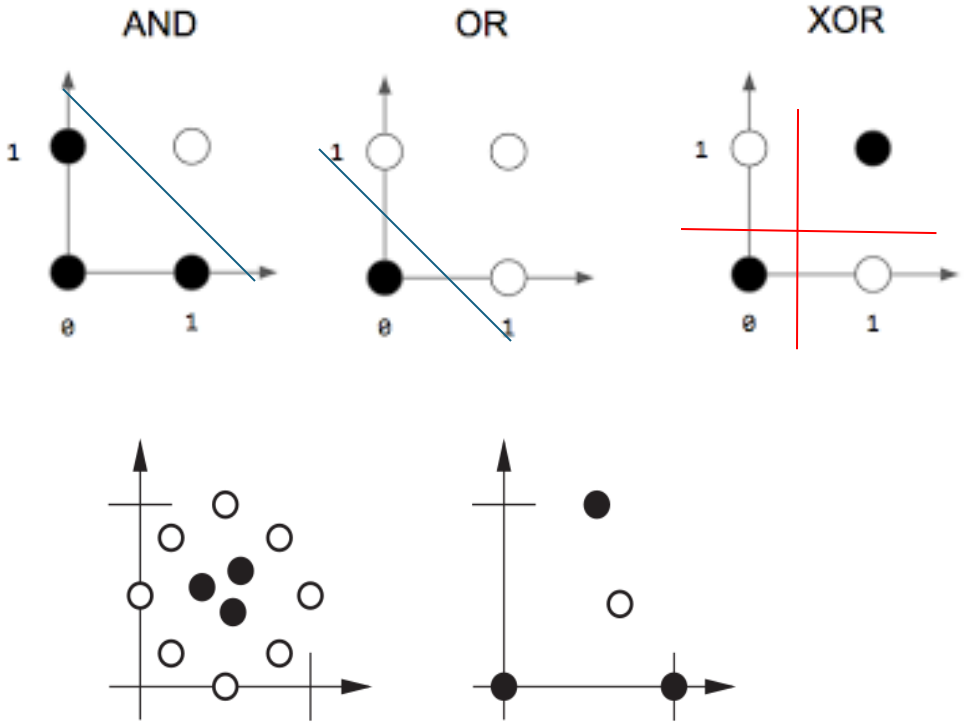
$$b \leftarrow b + e$$

If e = 1 then w' = w + x
If e = -1 then w' = w – x
If e = 0 then w' = w



Limits of perceptron

- **Can only learn linearly separable problems**
- **Cannot solve XOR**



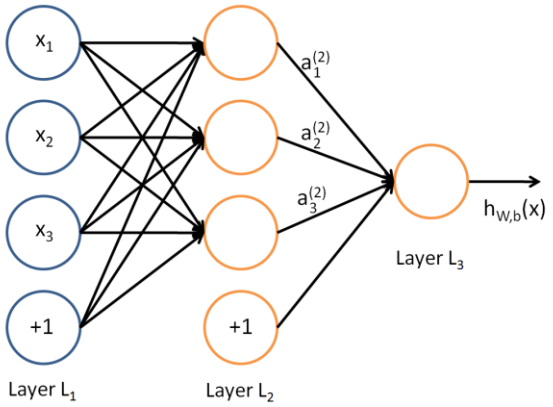
The missing ingredient:
hidden layers+ activation functions

Even though one perceptron can't represent XOR, multiple perceptrons arranged in layers can.

Multiple layers + nonlinear activation functions → ability to learn nonlinear patterns

Multi-layer Perceptron (MLP)

- A Multilayer Perceptron (MLP) is a neural network made by connecting multiple perceptron layers.
- The leftmost layer of the network is called the **input layer**, and the rightmost layer the **output layer**
- The middle layer of nodes is called the **hidden layer**, because its values are not observed in the training set.
- We also say that our example neural network has 3 **input units** (not counting the bias unit), 3 **hidden units**, and 1 **output unit**.



The circles labeled "+1" are called **bias units**, and correspond to the intercept term.

Class representation

- Binary classification: Two classes
  - Class 0: Not spam, Class 1: Spam
  - The labels are represented as: y=[0,1,1,0]

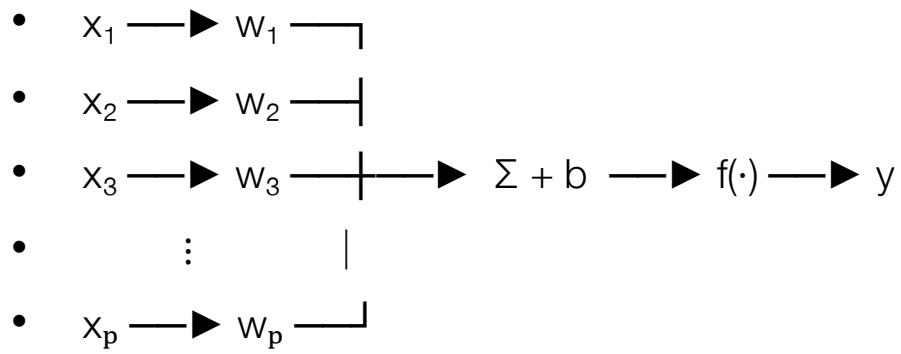
- Multiclass classification: More than two classes
  - Class 0: Cat, Class 1: Dog, Class 2: Bird.
    - A. Integer encoding (mlpclassifier in scikit-learn)
      - The target vector: y=[0,1,2,0]
    - B. One-hot encoding
      - Cat: [1,0,0], Dog: [0,1,0], Bird: [0,0,1]

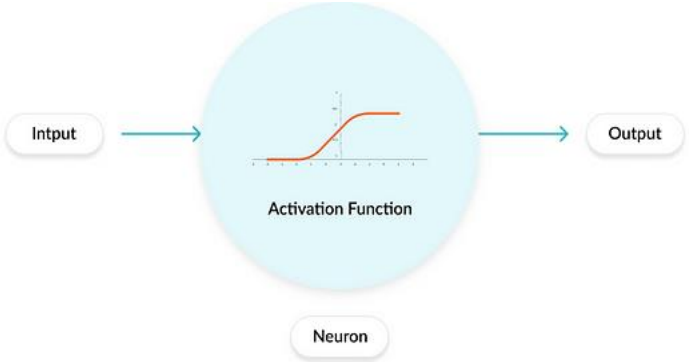


Activation Functions

Activation Function

- Activation function f:
  - converts the weighted sum into an output
  - Introduces nonlinearity.
- Without nonlinear activation functions, stacking layers of neurons would still produce only a linear transformation.





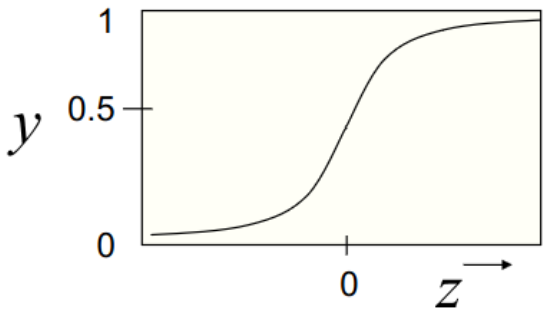
Sigmoid Function

- is used for binary classification tasks
- Turns the Logit into a Probability
  - We can interpret y(x) as the probability that the label of x is 1.

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

$$-\infty < z < \infty$$

$$0 < \sigma(z) < 1$$



The name “sigmoid” means “S-shaped”

Softmax Function

- The softmax function is typically applied to the output layer for multiclass classification, converting the model's outputs into a probability distribution across the classes.
- Softmax converts the vector of logits into a probability distribution:

$$P_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

$$[z_1, z_2, \ldots, z_K] \rightarrow \text{Softmax}$$

Sigmoid vs. Softmax

- We apply these functions to get a probability

Binary classification

Input → Linear → Logit → Sigmoid → Probability

Multiclass classification

Input → Linear → Logits → Softmax → Probabilities

- Sigmoid: $\sigma(z)_i = \frac{1}{1+e^{-z_i}}$
- Softmax: $\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$

```
z₁ = 2.0  ---[ Sigmoid ]--->  y₁ ≈ 0.88
z₂ = 1.0  ---[ Sigmoid ]--->  y₂ ≈ 0.73
z₃ = -1.0 ---[ Sigmoid ]--->  y₃ ≈ 0.27
```

```
z₁ = 2.0  \
z₂ = 1.0   ---[ Joint Softmax ]--->  y₁ ≈ 0.66 \
z₃ = -1.0 /                          y₂ ≈ 0.24  > Sum of outputs = 1.0
                                     y₃ ≈ 0.10 /
```

Sigmoid: Treats every class independently as a separate binary decision (probability between 0 and 1).
Softmax: Normalizes outputs into a probability distribution across K classes where all outputs sum to 1.

Sigmoid and Softmax (recap)

- Activation functions for the output layer
- Sigmoid is used for binary classification methods where we only have 2 classes, while SoftMax applies to multiclass problems.
- Sigmoid represents the probability of belonging to *class1* (the probability of belonging to *class2 = 1 - P(class1)*).
  - Sigmoid produces independent probabilities (*not* constrained to sum to one)
- Softmax produces a probability distribution over all predicted classes (The probabilities sum will be 1)

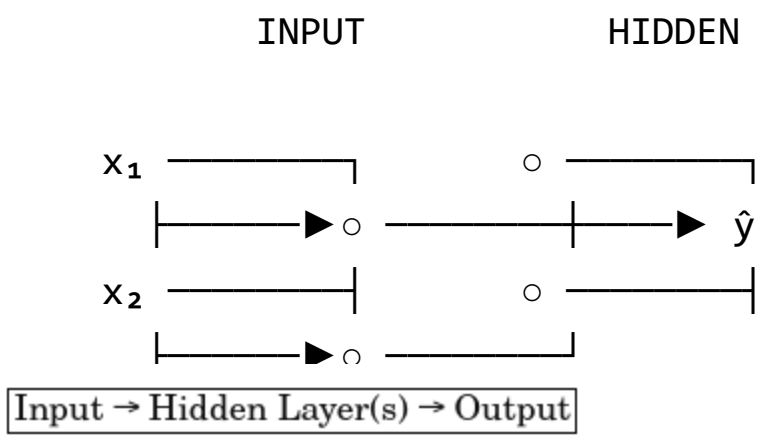


ANNs and their Architectures

Artificial Neural Network (ANN)

- **ANN**: Connect many artificial neurons together
- An ANN learns a mapping: $\mathbf{x} \rightarrow \hat{\mathbf{y}}$
  - x = input features
  - y^ = predicted output
  - weights and biases = learned parameters

- A **Multi-Layer Perceptron (MLP)** is a type of feed-forward ANN with an input layer, one or more hidden layers, and an output layer.



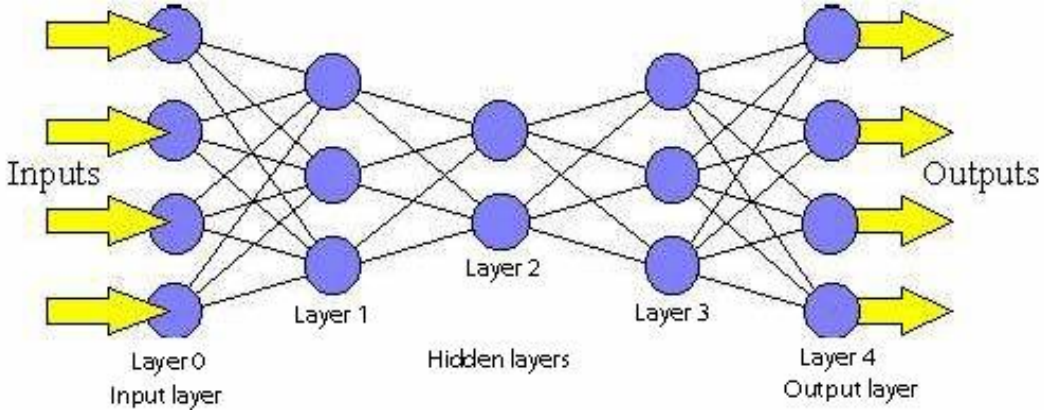
$$\mathbf{a}^{(1)} = f(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$\hat{\mathbf{y}} = g(\mathbf{W}^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)})$$

With **nonlinear activation functions**, multiple layers can learn complex nonlinear mappings.

Feed-forward networks

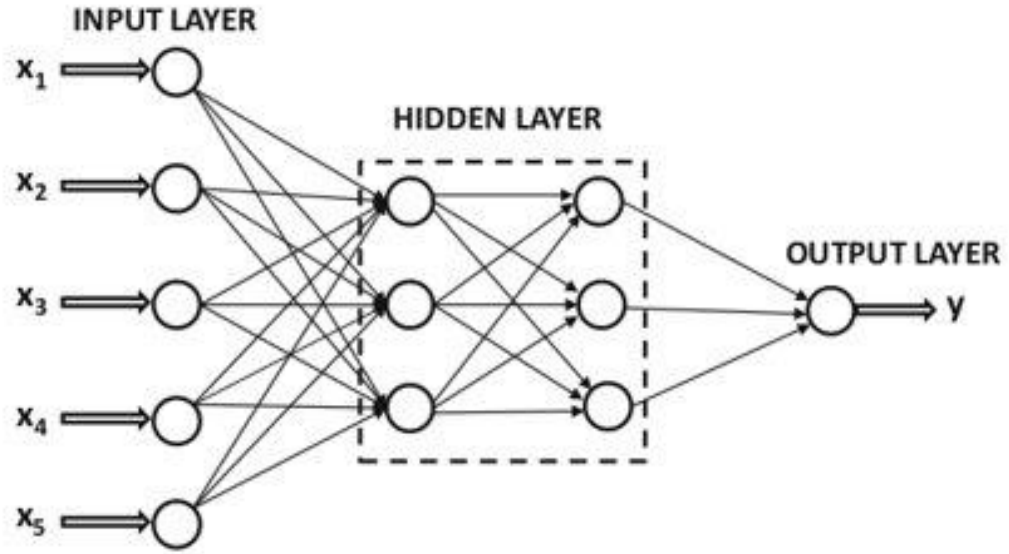
- The specific architecture of multilayer neural networks is referred to as feed-forward networks, because **successive layers feed into one another in the forward direction from input to output**.
- The default architecture of feed-forward networks assumes that all nodes in one layer are connected to those of the next layer.
- MLP is an FNN.



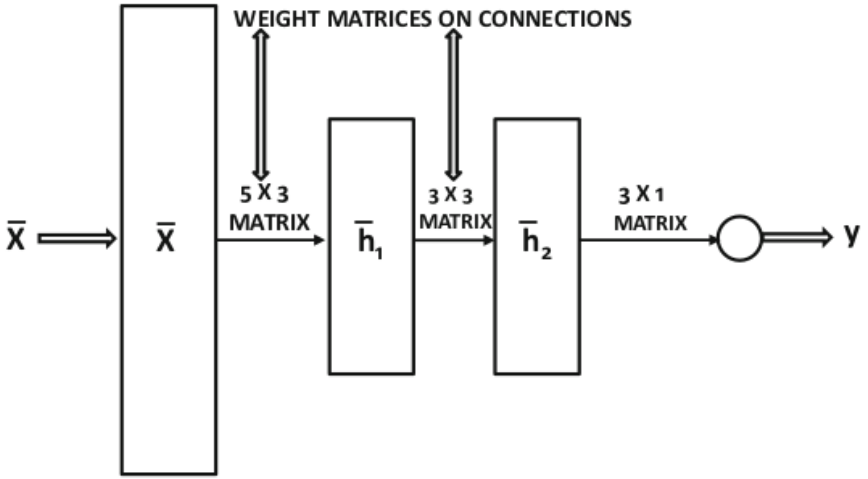
- Other types:
  - CNN, RNN, Transformer, Autoencoder, GAN, GNN

Architecture of feed-forward network

$$\overline{h}_1 = \Phi(W_1^T \overline{x}) \qquad \text{[Input to Hidden Layer]}$$
$$\overline{h}_{p+1} = \Phi(W_{p+1}^T \overline{h}_p) \quad \forall p \in \{1 \ldots k-1\} \qquad \text{[Hidden to Hidden Layer]}$$
$$\overline{o} = \Phi(W_{k+1}^T \overline{h}_k) \qquad \text{[Hidden to Output Layer]}$$



- Activation functions for **hidden** layers:
  - Rectified Linear Activation (ReLU), Logistic (Sigmoid), Hyperbolic Tangent (Tanh)
- Activation functions for **output** layers:
  - Linear, Logistic (Sigmoid), Softmax



Vector notation for binary classification



Loss and Cost Functions

Loss Function

- The problem: A neural network makes predictions.
- Question: How do we know whether the prediction is good or bad?
- Example:
  - Actual value: 10
  - Model prediction: 7
  - Prediction error: 3
  - We need a way to measure how wrong the model is.
  - ➡️ This is the job of a loss function.

What is Loss

- Loss = How wrong was one prediction?
- Better prediction → smaller loss
- Worse prediction → larger loss
- The ideal loss is usually: 0

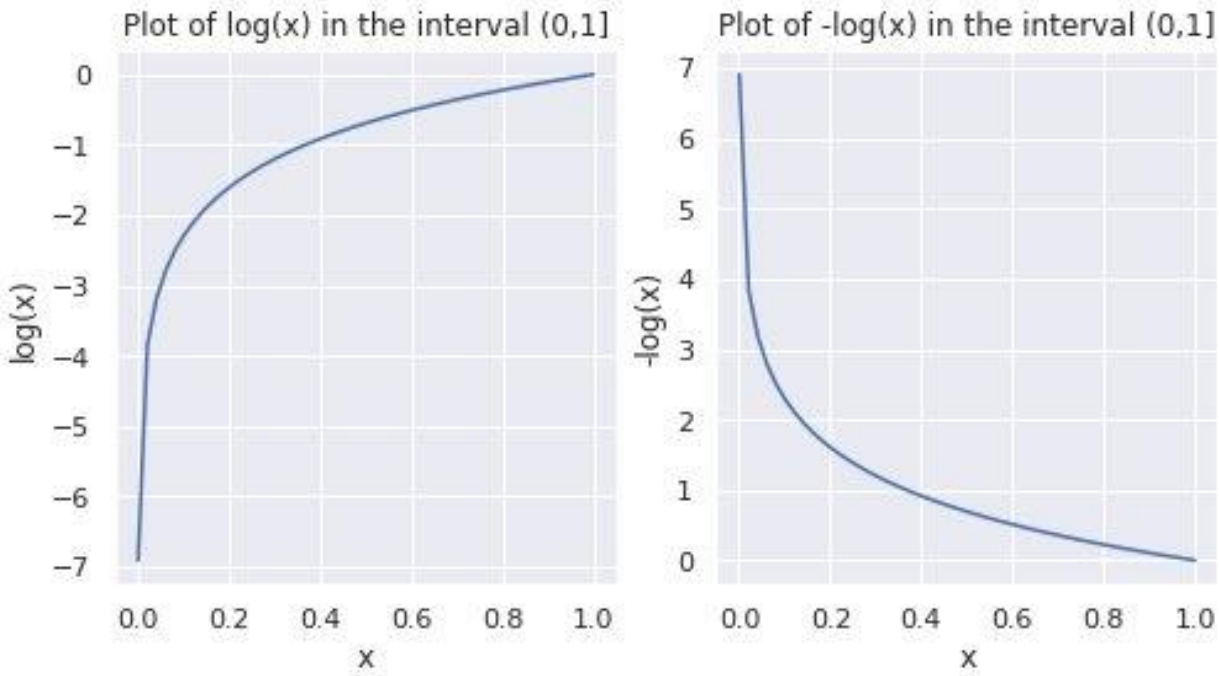
Common Loss: Mean Squared Error

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- y = actual value
- $\hat{y}$ = predicted value
- L = loss

Cross-Entropy Loss

- Cross-entropy loss is a function that measures how far a neural network's predicted probabilities are from the actual class labels.
- Cross-entropy loss is one of the most common loss functions for classification problems.
- It measures how close the model's predicted probabilities are to the correct answer.
- Given a true distribution t and a predicted distribution p, the cross entropy between them is given by the following equation:



$$H(\mathbf{t}, \mathbf{p}) = -\sum_{s \in S} \mathbf{t}(s).log(\mathbf{p}(s))$$

Cross entropy loss

- For multi-class classification:

$$\mathbf{z} = [z_1, z_2, \ldots, z_K]$$

$$\mathbf{p} = \text{softmax}(\mathbf{z})$$

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

$$\mathbf{y} = [0, 0, 1, 0]$$

$$L = -\sum_k y_k \log p_k$$

- For binary classification:

$$y \in \{0, 1\}$$

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$L = -\left[y \log(p) + (1 - y) \log(1 - p)\right]$$

- p is the probability of one
- 1-p is the probability of zero

Loss vs. Cost Function

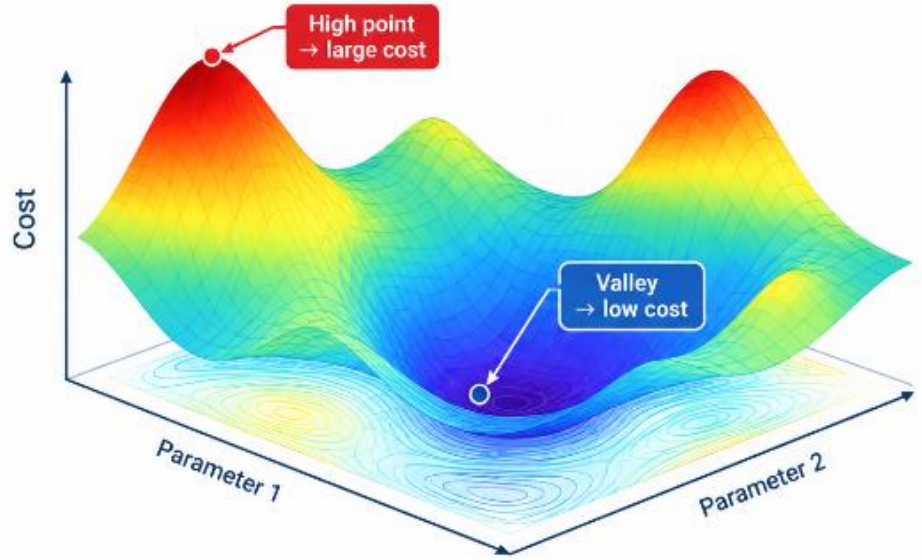
- Loss: Measures the error for one training example: L_i
- Cost function: Measures the overall error across a dataset.

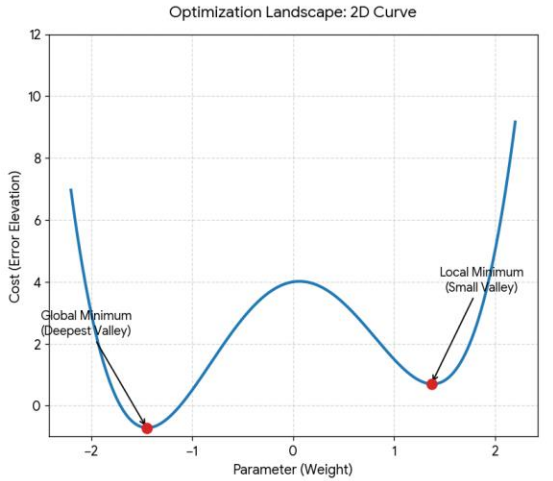
$$J(\theta) = \frac{1}{n} \sum_{i=1}^{n} L_i$$

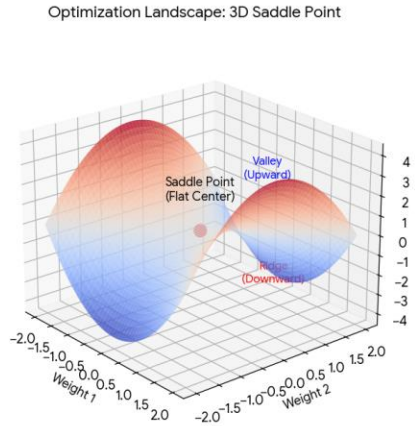
- Individual example = **individual mistake** $\longrightarrow$ Loss
- Whole training set = **overall performance** $\longrightarrow$ Cost

Cost Function as a Landscape

- Imagine the cost function as a landscape.
  - A **local minimum** is a point where the cost is lower than all surrounding points, but it is not the lowest possible point on the entire landscape
  - A **saddle point** gets its name because the math resembles a horse saddle or a mountain pass. At this exact point, the slope is zero, but it is a minimum from one direction and a maximum from another.
- Training tries to move the model downhill.
  - This is the intuition behind gradient descent.







Loss Function vs. Accuracy

- Accuracy asks:
  - "How many predictions were correct?"
- Loss asks:
  - "How wrong were the predictions, and how confident was the model?"
- A model can have the same accuracy but different losses.
- loss contains more information about the model's predictions.
- Accuracy: useful for evaluation.
- Loss: useful for training.



Regularization

Regularization

- Regularization techniques help prevent overfitting by imposing constraints on the model's parameters
- Overfitting happens when a model learns the training data too well, including the noise and outliers, which causes it to perform poorly on new data.
- Regularization adds a penalty to the model for being too complex, encouraging it to stay simpler and more general.

Regularization

- Regularization adds an additional term to the objective that discourages undesirable model behavior.
  - Objective = Loss + Regularization

$$\min_f [\sum_{i=1}^{n} (y_i - f(x_i))^2 + \lambda J(f)].$$

- $\sum (y_i - f(x_i))^2$ = **data-fitting term**
- $J(f)$ = **penalty function**
- λ = **regularization parameter**
- $\lambda J(f)$ = **regularization term**

  - Regularization= λ×Penalty
    - When λ is small, the penalty has little influence.
      - Better fit to training data
      - Greater risk of overfitting
    - When λ is large, the penalty becomes important.
      - The model therefore prefers a smoother, simpler function.
      - Lower risk of overfitting

Regularization

- **L1 regularization** (LASSO) leads to sparse models by adding a penalty based on the absolute value of coefficients.

$$Loss_{L1} = Loss + \lambda \sum_{j=1}^{p} |w_j|$$

- **L2 regularization** (Ridge) encourages smaller, more evenly distributed weights by adding a penalty based on the square of the coefficients.

$$Loss_{L2} = Loss + \lambda \sum_{j=1}^{p} w_j^2$$

- **Elastic Net:** combines L1 and L2

$$Loss_{EN} = Loss + \lambda \left[\alpha \sum_{j} |w_j| + (1-\alpha) \sum_{j} w_j^2 \right]$$



Optimization and training

Optimization

- Machine learning optimization is the process of finding the model parameters that minimize a loss (or cost) function.
  - update the θ values to reach the best value that minimizes the error between the predicted value and the true value through a loss/cost function.

$$\theta^* = \arg\min_{\theta} L(\theta)$$

Training

- A neural network's parameters:
  - Weights W
  - Biases b
- These parameters determine its predictions.
- Training tries to find parameters that produce a small cost.
  - Find W,b that minimize J(W,b)
- **Goal:** Minimize Cost

Epoch vs iteration

- An **epoch** is **one complete pass through the entire training dataset**.
- **Iteration** is the number of batches or steps through partitioned packets of the training data, needed to complete one epoch.
  - An **iteration** is **one parameter update**.
- a **batch** is a group of training examples processed together
  - The number of iterations = the number of batches needed to complete one epoch.

$$\text{iterations per epoch} = \frac{\text{number of training examples}}{\text{batch size}}$$

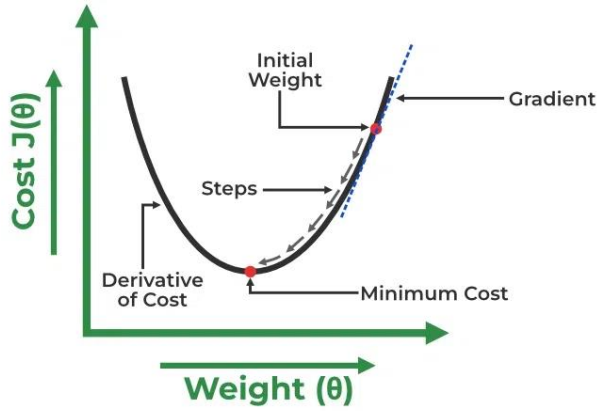


Gradient Descent

Gradient Descent

- "Which direction should I change my parameters to reduce the cost?"
- Gradient descent uses the gradient: $\nabla J(\theta)$ $\frac{\partial L}{\partial w}$
- Optimizer updates the parameters:

$$\theta_{new} = \theta_{old} - \alpha \nabla J(\theta) \qquad w_{new} = w - \alpha \frac{\partial L}{\partial w}$$



- θ = model parameters
- α = learning rate
- ∇J = gradient of the cost

Gradient descent algorithm

Algorithm Gradient Descent

Input: initial parameters $\Theta^{(0)}$, number of iterations T, learning rate α
Output: final learning $\Theta^{(T)}$

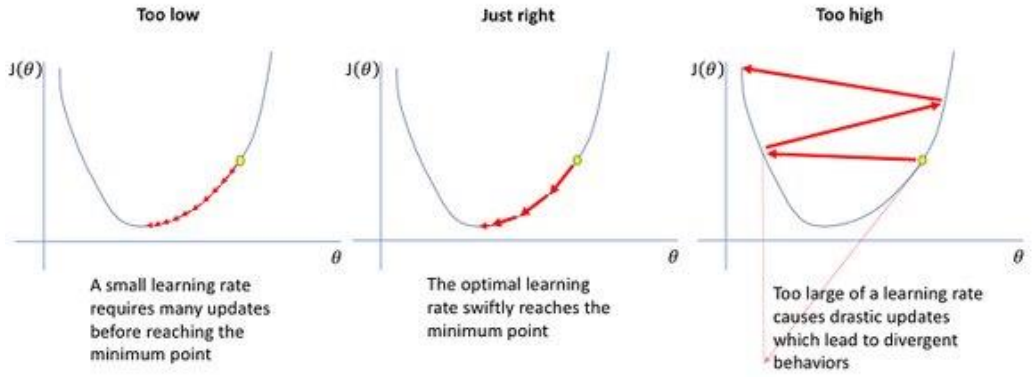
1. **for** $t = 0$ **to** $T - 1$
2. estimate $\nabla \mathcal{L}(\Theta^{(t)})$
3. compute $\Delta\Theta^{(t)} = -\nabla \mathcal{L}(\Theta^{(t)})$
4. $\Theta^{(t+1)} := \Theta^{(t)} + \alpha\Delta\Theta^{(t)}$
5. **return** $\Theta^{(T)}$

$$\theta_j = \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

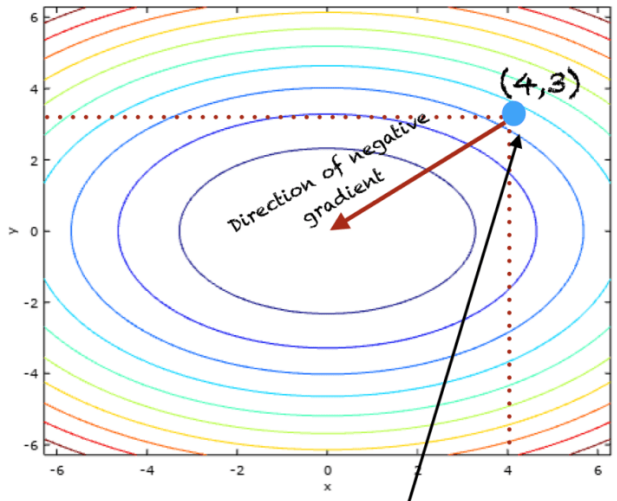
Gradient descent can be performed in three main forms: full-batch, stochastic, and mini-batch gradient descent.

Learning rate

- The learning rate (=size of steps), denoted by the symbol **α or η**, is a hyper-parameter used to govern the **pace** at which an algorithm updates or learns the values of a parameter estimate.
  - Smaller learning rates
    - more training epochs because of the fewer changes.
    - Training may take a long time to converge.
    - **Risk:** With a fixed training budget, the model may not have enough time to reach a good solution.
  - larger learning rates result in faster changes.
    - Training may progress faster initially.
    - frequently result in a suboptimal final set of weights.
    - **Risk:** The loss may oscillate, increase, or fail to converge.



Example



$$J(w_1, w_2) = w_1^2 + 2w_2^2$$

$$\boxed{w_1 = 3, \quad w_2 = 4}$$

$$\boxed{\eta = 0.1}$$

Step 1: Calculate the gradient

We have:

$$\frac{\partial J}{\partial w_1} = 2w_1$$

$$\frac{\partial J}{\partial w_2} = 4w_2$$

At $(3, 4)$:

$$\nabla J = \begin{bmatrix} 2(3) \\ 4(4) \end{bmatrix} = \begin{bmatrix} 6 \\ 16 \end{bmatrix}$$

Step 2: Update the weights

Gradient descent:

$$w_i^{new} = w_i^{old} - \eta \frac{\partial J}{\partial w_i}$$

For w_1:

$$w_1^{new} = 3 - 0.1(6) = 2.4$$

For w_2:

$$w_2^{new} = 4 - 0.1(16) = 2.4$$

So after the **first iteration**:

$$\boxed{(w_1, w_2) = (2.4, 2.4)}$$

Step 3: Calculate the new cost

Initially:

$$J(3, 4) = 3^2 + 2(4^2)$$

$$= 9 + 32 = \boxed{41}$$

After one update:

$$J(2.4, 2.4) = 2.4^2 + 2(2.4^2)$$

$$= 5.76 + 11.52$$

$$\boxed{J = 17.28}$$

So the cost decreased:

$$\boxed{41 \rightarrow 17.28}$$

Full Gradient Descent (batch GD)

- Calculates the gradient for each example
- only updates the model with the average gradient after all training examples have been evaluated.
- time-consuming and computationally expensive.
- it is very slow on very large training data.

$$\nabla J(w) = \frac{1}{N} \sum_{i=1}^{N} \nabla L_i(w)$$

$$w_{\text{new}} = w - \eta \nabla J(w)$$

Stochastic Gradient Descent

- calculates the gradient using **one training example at a time**, then immediately updates the weights.
- This modification of GD can **reduce the computational time** significantly.
- making it much **faster** as there is much fewer data to manipulate at a single time, unlike Batch GD.
- this approach may lead to **noisier** results than the GD, because it iterates one observation at a time.

$$w \leftarrow w - \eta \nabla L_i(w)$$

Mini batch gradient descent

- split the dataset into **small subsets (mini batches)** and compute the gradients for each minibatch.
- For each minibatch, the mean gradient is computed and the weights are updated.
- Just like SGD, the average cost over the epochs in mini-batch gradient descent **fluctuates** because we are averaging a small number of examples at a time.

$$\nabla J_B(w) = \frac{1}{|B|} \sum_{i \in B} \nabla L_i(w)$$

$$w \leftarrow w - \eta \nabla J_B(w)$$

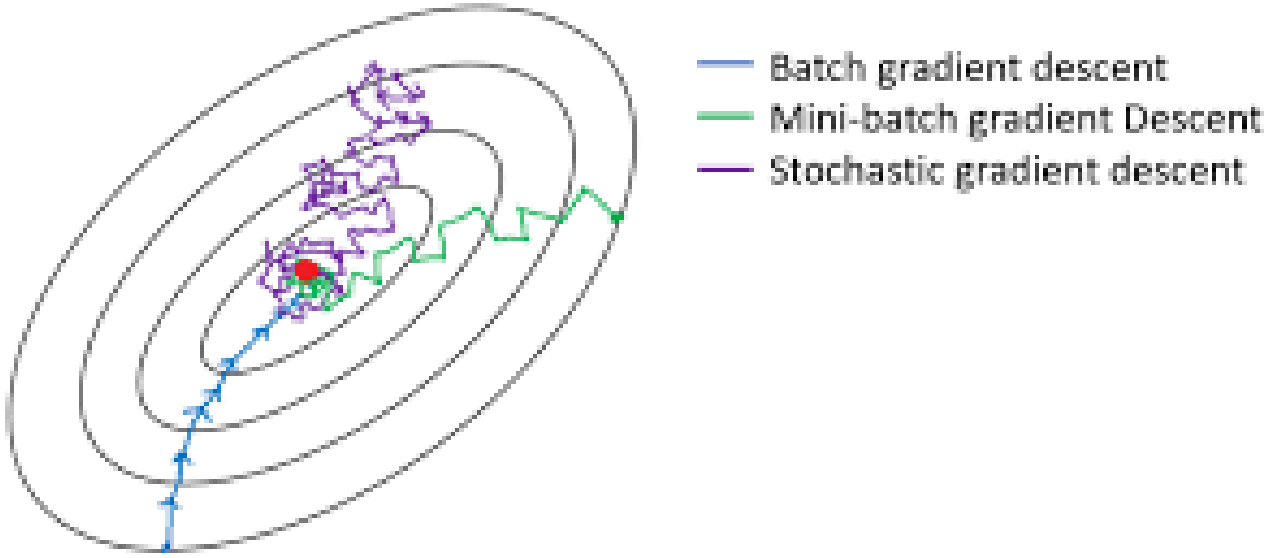


FULL-BATCH	SGD	MINI-BATCH
All data	1 example	Small batch
↓	↓	↓
Gradient	Gradient	Gradient
↓	↓	↓
Update	Update	Update
↓	↓	↓
Repeat	Repeat	Repeat

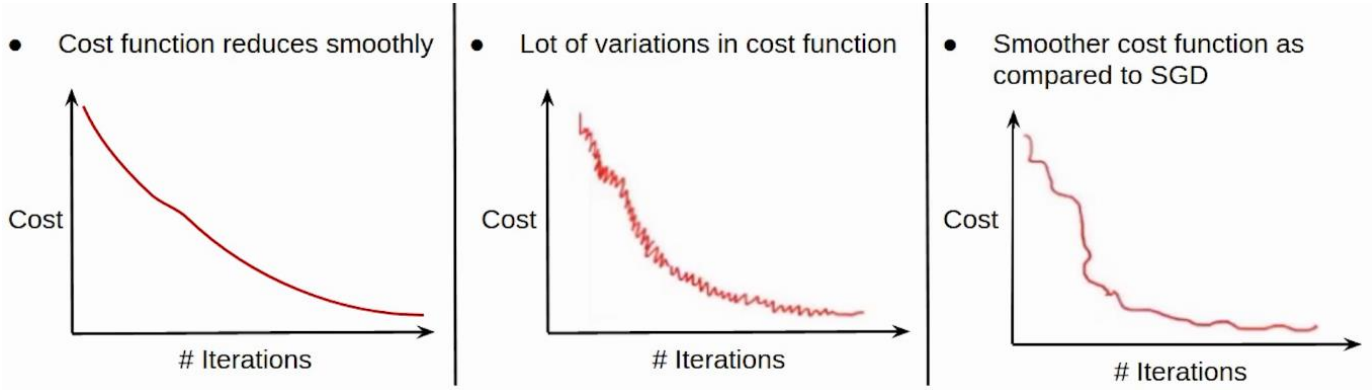
	Full-Batch GD	SGD	Mini-Batch GD
Examples per update	N	1	B
Updates per epoch	1	N	N/B
Computation per update	High	Very low	Moderate
Typical modern practice	Less common	Less common alone	Most common

Batch size = B, N = # training examples

Comparison



	Full-Batch GD	SGD	Mini-Batch GD
Gradient	Stable	Noisy	Moderately noisy





Backpropagation and Chain rule

Backpropagation

- Backpropagation provides an efficient method for computing gradients of the loss with respect to network parameters.
- It contains two main phases, referred to as the forward and backward phases, respectively.
  - The **forward phase** is how the input moves through the neural network to produce a prediction and compute Loss.

$$\boxed{\text{Input} \rightarrow \text{Prediction} \rightarrow \text{Loss}}$$

  - the **backward phase** is required to compute gradients for all parameters and updating parameters

$$\boxed{\text{Loss} \rightarrow \text{Gradients} \rightarrow \text{Parameters}}$$

Backward phase

- The main goal of the backward phase is to learn the **gradient of the loss function** with respect to the different weights by using the ***chain rule*** of differential calculus.
  - Backpropagation uses the chain rule to calculate the gradients of the weights in each layer,
  - Gradient Descent uses these gradients to update the weights and biases and minimize the loss.

$$z = \mathbf{w}^\top \mathbf{x} + b$$

$$a = f(z)$$

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial a}\frac{\partial a}{\partial z}\frac{\partial z}{\partial w_i}$$

$$\frac{\partial z}{\partial w_i} = x_i$$

$$\boxed{\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial a} f'(z) x_i}$$

- Since these gradients are learned in the backward direction, starting from the output node, this learning process is referred to as the backward phase.

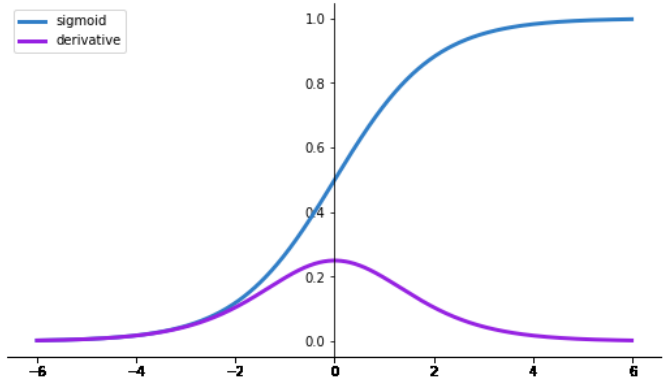


Activation Functions and Their Properties in Backpropagation

Sigmoid

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

- It is derivable at every point (gradients can be computed)
- It saturates and kills gradients (vanishing gradient)
  - problematic in chain rule of backpropagation
- Not zero centered.
  - This will obstruct the possible update directions of the gradients.
  - it would take a substantially longer time to converge.
  - It is computationally expensive because of the exponential term in it.



$$\frac{d\sigma(z)}{dz} = \frac{e^{-z}}{(1+e^{-z})^2}$$

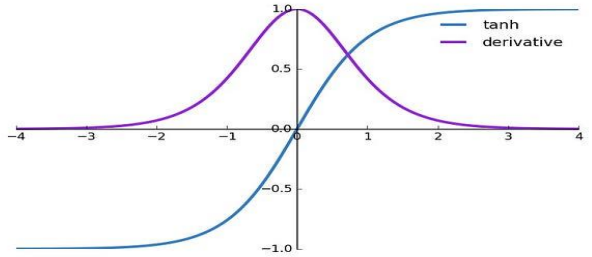
The derivative can be written in terms of the activation output itself, which makes them convenient during backpropagation

$$\sigma'(z) = \sigma(z)(1-\sigma(z))$$

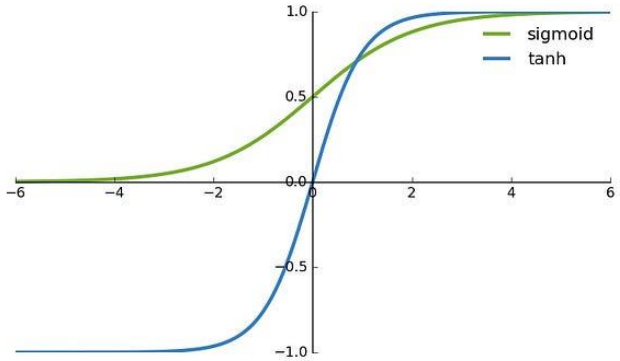
Hyperbolic Tangent

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

- This is an extension of logistic sigmoid
  - Is a mathematically shifted version of sigmoid and works better than sigmoid in most cases.
- Zero centered (symmetry around zero)
  - improve the convergence of the backpropagation algorithm.
- vanishing gradients (gradients saturation) similar to the sigmoid activation function.
  - But the derivatives are steeper than that of the sigmoid. Hence making the gradients stronger for tanh than sigmoid.
- Computationally expensive



$$\frac{d}{dz}\tanh(z) = 1 - \tanh^2(z)$$



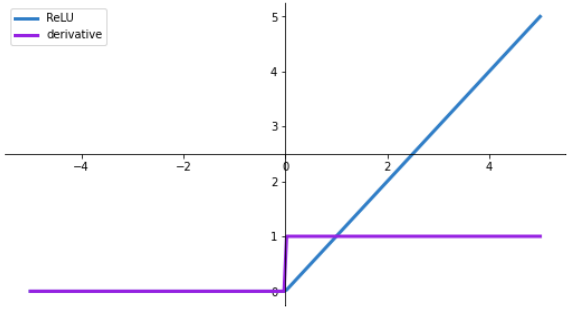
The derivative can be written in terms of the activation output itself, which makes them convenient during backpropagation

$$\frac{\partial a}{\partial z} = 1 - a^2$$

ReLU (Rectified Linear Unit)

- It gives an output x if x is positive and 0 otherwise.
- It is derivable, but It is non-differentiable at 0.
- It suffers from the dying ReLU problem (when output is negative).
  - ReLU is always going to discard the negative values
- ReLU is less computationally expensive than tanh and sigmoid because it involves simpler mathematical operations.

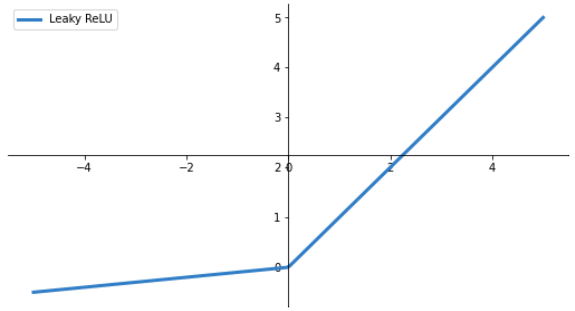
$$f(x) = max(0, x).$$



Leaky ReLU

$$f(x) = \max(\alpha x, x)$$

- α is a small constant (normally 0.01).
- Advantages:
  - If the input is negative, the gradient will be α. As a result, there will be learning for these units as well.
- Disadvantage:
  - the value of α is always constant and is a hyperparameter (parametric ReLU).

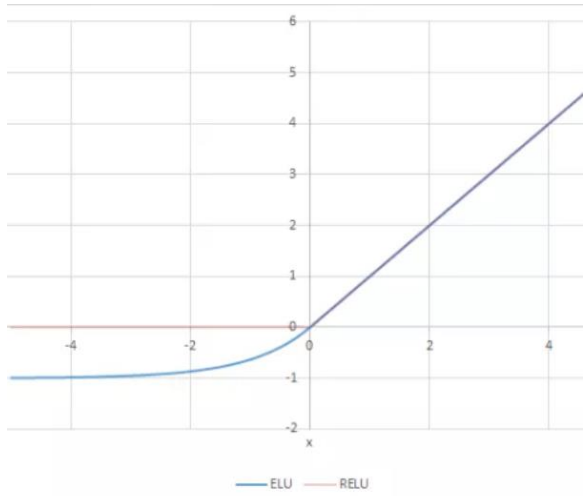


ELU (Exponential Linear Unit)

- Advantages:
  - Unlike ReLU, It is derivable at 0.
  - ELU becomes smooth slowly
    - This smoothness plays a beneficial role in optimization and generalization.
  - Avoids dead ReLU problem
- Disadvantages:
  - It is computationally expensive due to the presence of the exponential term.
  - α is a hyperparameter

$$ELU$$

$$\begin{cases} x & for\ x \geqslant 0 \\ \alpha\left(e^{x}-1\right) & for\ x < 0 \end{cases}$$

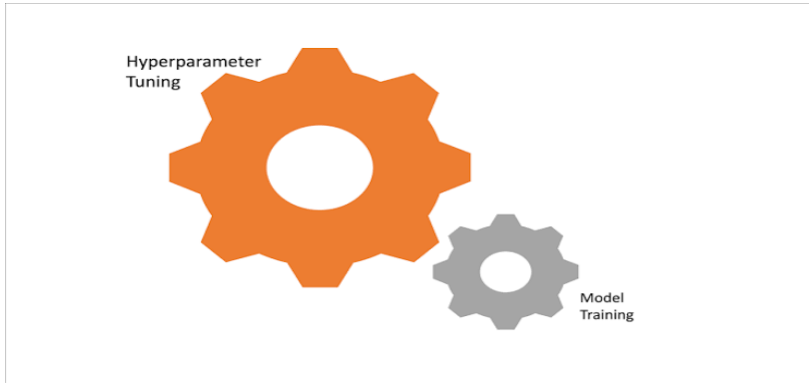




Hyperparameter Tuning and Validation

Hyperparameter tuning

- Hyperparameter tuning is a process of selecting the optimal values for hyperparameters of the machine learning model.
- It is an important step in the model development process, as the choice of hyperparameters can have a significant impact on the model's performance.





Parameters vs hyperparameters

- **Parameters:** learned automatically during training.
  - Define the model's learned behavior
  - must be determined using the training data
  - Example: weights in a neural network
- **Hyperparameters:** selected before or during training.
  - Hyperparameters control how a machine-learning model learns.
  - Proper tuning can significantly improve model performance.
  - Must be determined using validation data
  - Examples: Learning rate, Batch size, Number of layers, Tree depth

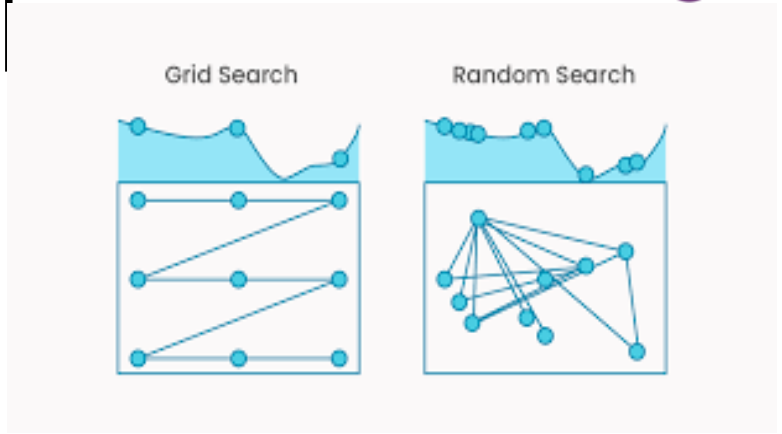
Why Tune Hyperparameters?

- Poor hyperparameter choices can lead to:
  - **Underfitting** → model is too simple
  - **Overfitting** → model memorizes the training data
- Slow or unstable training
- Poor generalization
- **Objective:** maximize validation performance while avoiding unnecessary model complexity.

Common Tuning Strategies

- Grid Search
  - Tests every combination from a predefined grid.
  - Simple but computationally expensive.
- Random Search
  - Randomly samples combinations.
  - Often more efficient than grid search.
- Bayesian Optimization
  - Uses previous evaluations to select promising configurations.
  - **use probability to reason about which hyperparameters are likely to give good results**.

Grid Search vs. Random Searc



- **Grid Search**
  - Exhaustive search. Try everything in the grid.
- **Random Search**
  - Explore the space intelligently by sampling configurations.
- Example:
  - Learning rate: 0.001, 0.01, 0.1
  - Batch size: 16, 32, 64
  - Number of layers: 2, 3, 4
- Grid search → **27 combinations**
- Random search → evaluate a smaller number of randomly selected combinations.

Validation Strategy

- **How do we evaluate each configuration?**
- Split data into:
  - Training set
  - Validation set
  - Test set
- Train the model using the training set.
- Use the validation set to select hyperparameters.
- Use the test set **only for final evaluation**.

Cross-validation (CV)

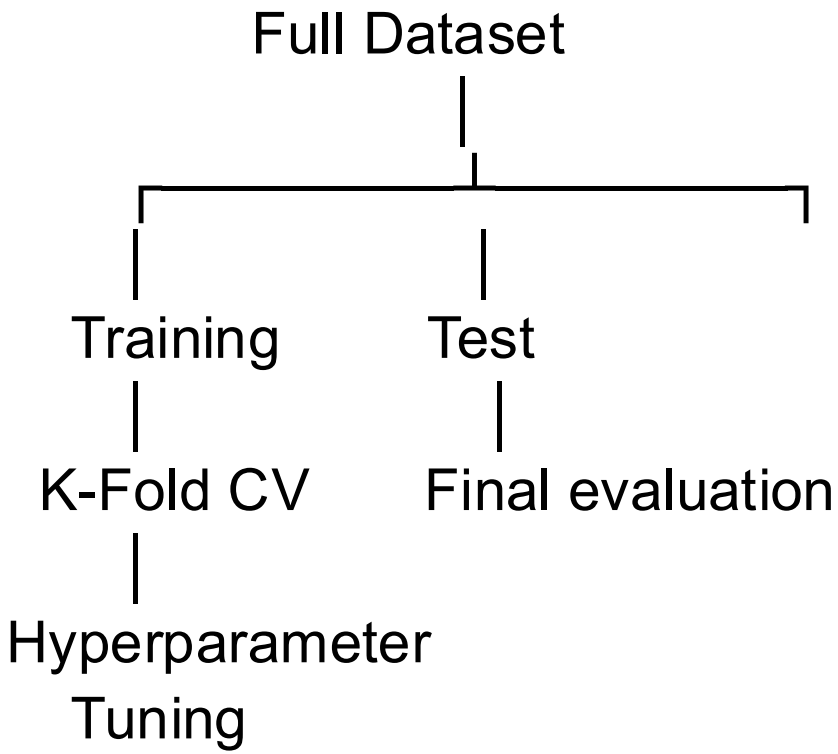
- is a technique used in machine learning to assess the generalization capability and performance of a machine learning model.
- This involves creating **multiple subsets of datasets called folds** and iteratively performing training and evaluation of models on different training and testing datasets each time.
- Cross-validation
  - K-fold
  - Leave-one-out
  - Stratified k-fold

K-fold cross validation

- **A robust approach for model evaluation and hyperparameter tuning**

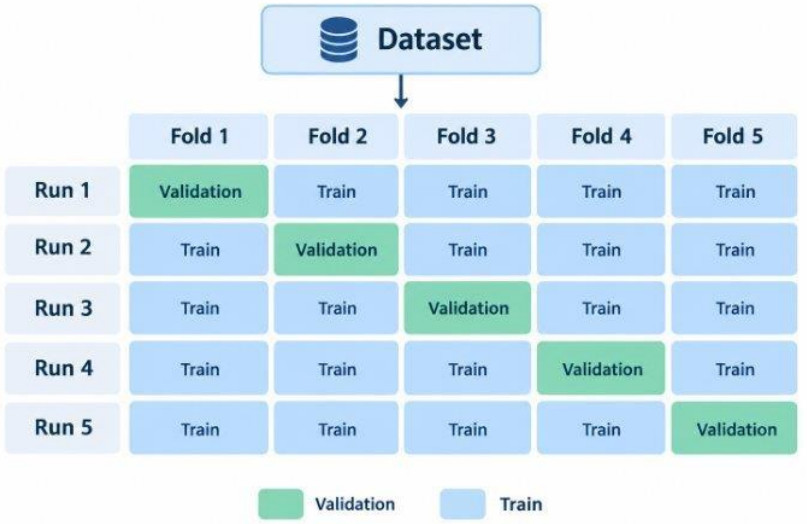
- K-Fold Cross-Validation divides the dataset into **K equally sized folds**.
- Each fold is used as the validation set exactly once.
- The model is trained and validated **K times**.
- Final performance is typically calculated as the **average across all K folds**.

Keep the Test Set Separate



The test set should remain untouched during hyperparameter tuning.

K-fold cross validation



- **For each hyperparameter configuration:**
  - Split the training data into K folds.
  - Train on K–1 folds.
  - Validate on the remaining fold.
  - Repeat K times.
  - Calculate the mean validation score.
  - Select the configuration with the best average score.

- The **Leave-One-Out (LOO) method** is a special case of **cross-validation**, where **one data point is left out for validation at a time**, and all remaining data points are used for training.

Choosing K

- **Typical choice: 5-fold or 10-fold cross-validation**
  - **K = 5** → good balance between computational cost and reliable evaluation.
  - **K = 10** → potentially more robust, but more computationally expensive.
- **K = 2–3** → useful when computational resources are limited.
- K= N → LOO can be used for small datasets

Stratification

- For classification problems, we often have different classes.
  - Class 0: 800 observations
  - Class 1: 200 observations
- A random fold might accidentally contain:
  - Fold 1: 95% Class 0 / 5% Class 1
  - Fold 2: 72% Class 0 / 28% Class 1
  - This can make our performance estimates unstable
- Stratified cross-validation attempts to preserve the **class proportions in every fold**.

Regular K-Fold

```
Fold 1 → 90% Class 0 / 10% Class 1
Fold 2 → 75% Class 0 / 25% Class 1
Fold 3 → 85% Class 0 / 15% Class 1
```

Stratified K-Fold

```
Fold 1 → 80% Class 0 / 20% Class 1
Fold 2 → 80% Class 0 / 20% Class 1
Fold 3 → 80% Class 0 / 20% Class 1
Fold 4 → 80% Class 0 / 20% Class 1
Fold 5 → 80% Class 0 / 20% Class 1
```



Classification metrics

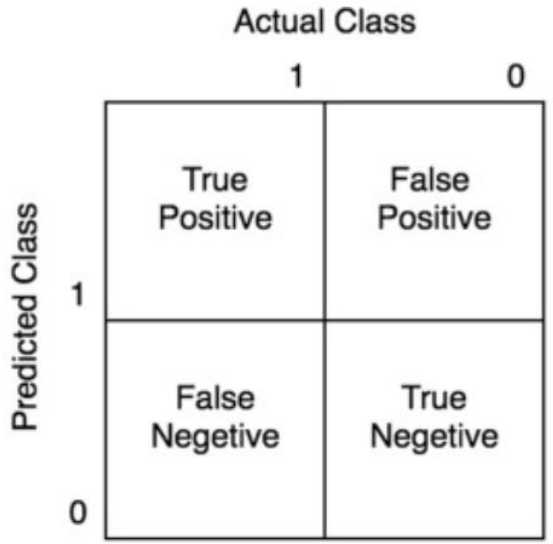
Binary classification example

- In a medical diagnosis, a binary classifier for a specific disease could take a patient's symptoms as input features and predict whether the patient is healthy or has the disease.
- The possible outcomes of the diagnosis are **positive** and **negative**.

Application	Observation	0	1
Medical Diagnosis	Patient	Healthy	Diseased
Email Analysis	Email	Not Spam	Spam
Financial Data Analysis	Transaction	Not Fraud	Fraud
Marketing	Website visitor	Won't Buy	Will Buy
Image Classification	Image	Hotdog	Not Hotdog

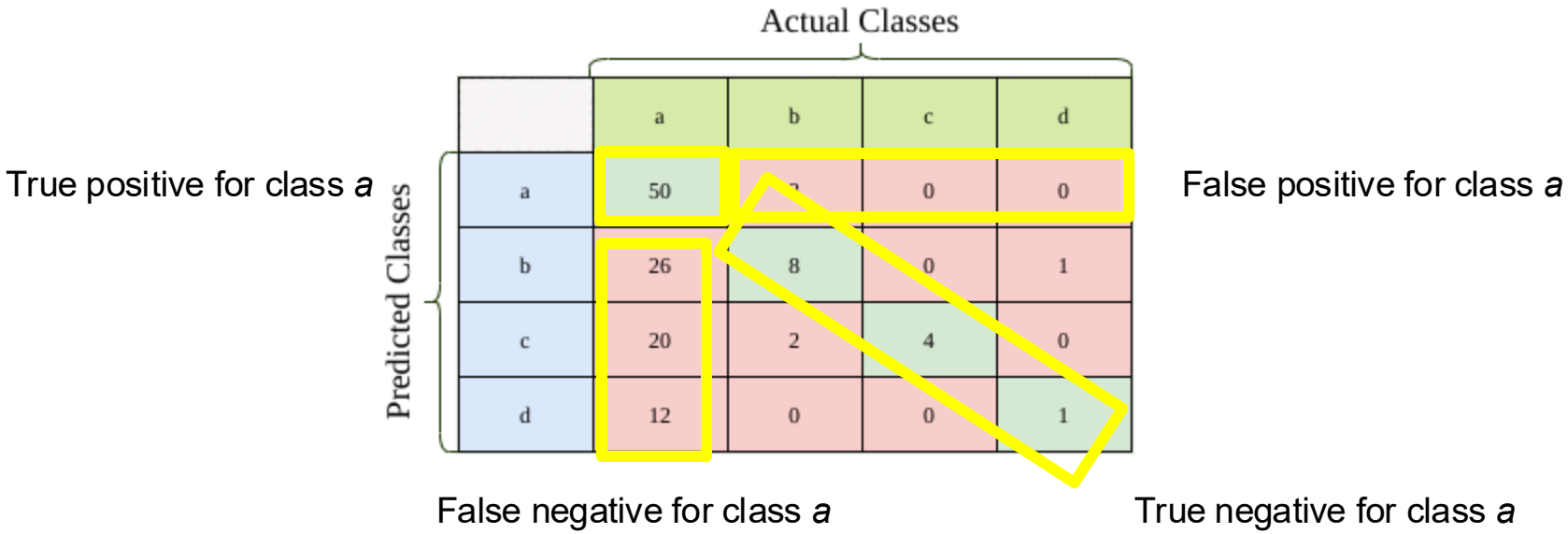
Confusion matrix

- Confusion matrices represent counts from predicted and actual values.
- "TP" = the number of positive examples classified accurately.
  - Observation is predicted positive and is actually positive.
- "TN" = the number of negative examples classified accurately.
  - Observation is predicted negative and is actually negative.
- "FP" = the number of actual negative examples classified as positive;
  - Observation is predicted positive and is actually negative.
- "FN" = the number of actual positive examples classified as negative.
  - Observation is predicted negative and is actually positive.



Confusion matrix for multiclass classification

- When we are working on label *a*, only label a is positive and all the other labels are negative.

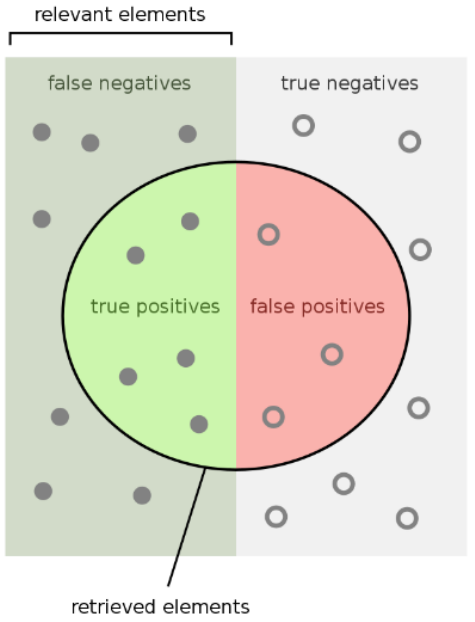


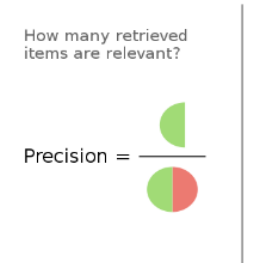
Precision

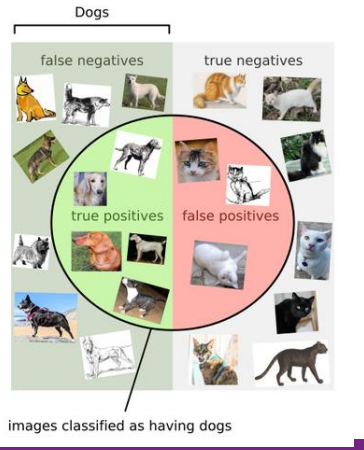
- Precision or the positive predictive value, is the fraction of positive values out of **the total predicted positive** instances.
- precision is the proportion of positive values that were correctly identified

$$\text{Precision} = \frac{TP}{TP+FP}$$

- precision tells you how much you can trust your classifier when it tells you an instance belongs to the positive class.







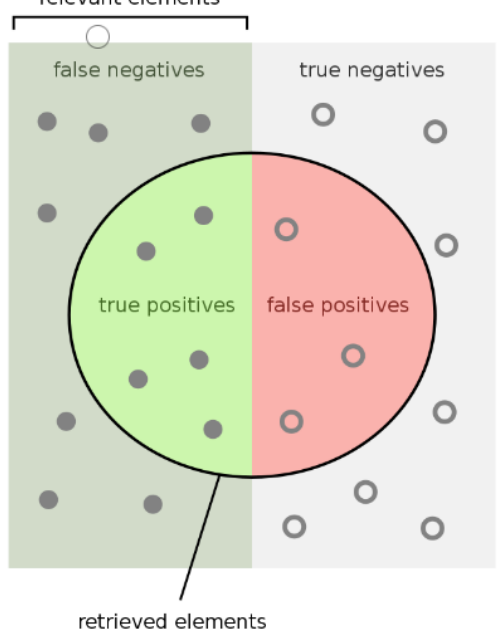
https://en.wikipedia.org/wiki/Precision_and_recall

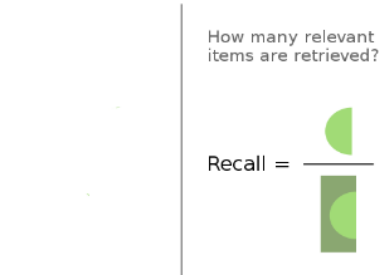
Recall (Sensitivity)

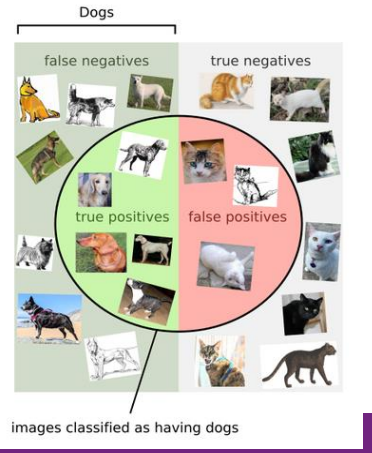
- Sensitivity, True positive rate (TPR), hit rate, or recall
- **correctly classified samples to the total number of positive samples**

$$\text{Sensitivity} = \frac{TP}{TP+FN} \qquad TPR = \frac{TP}{TP+FN} = \frac{TP}{P},$$

- the recall measures the model's ability to detect Positive samples.
- The higher the recall, the more positive samples detected.



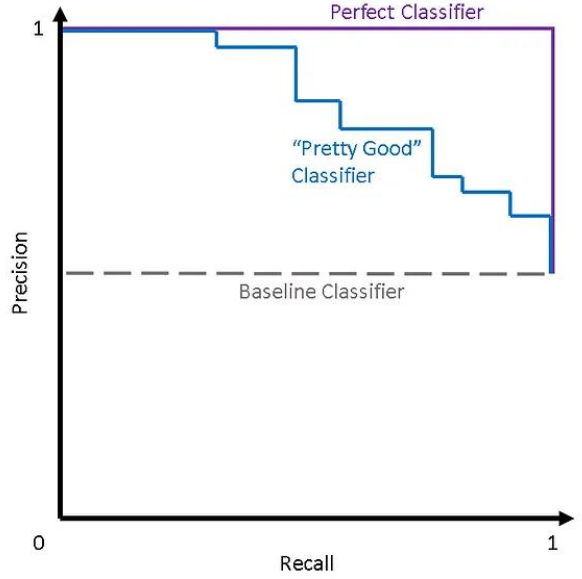




https://en.wikipedia.org/wiki/Precision_and_recall

Precision-recall curve

- Usually, increasing the value of precision decreases the value of recall, and vice-versa.
- A PR curve is simply a graph with Precision values on the y-axis and Recall values on the x-axis.
- A good classifier will maintain both a high precision and high recall across the graph, and will “hug” the upper right corner in the figure below.



Accuracy

- **Classification accuracy** is the proportion of predictions made by a classification model that are correct out of all predictions.

$$\text{Accuracy} = \frac{TN+TP}{TN+FP+FN+TP}$$

- Accuracy can be misleading if used with imbalanced datasets
  - With an imbalanced dataset, the majority class can dominate this calculation. A model can therefore achieve high accuracy simply by predicting the majority class most of the time.

$$Accuracy = \frac{0+990}{1000} = 0.99 \qquad Recall = \frac{0}{0+10} = 0$$

Specificity

- Specificity, True negative rate (TNR), or inverse recall
- Specificity gives the fraction of negative values out of the total actual negative instances.
- Specificity = Negative samples to the total number of negative samples

$$\text{Specificity} = \frac{TN}{TN+FP}$$

$$TNR = \frac{TN}{FP+TN} = \frac{TN}{N}$$

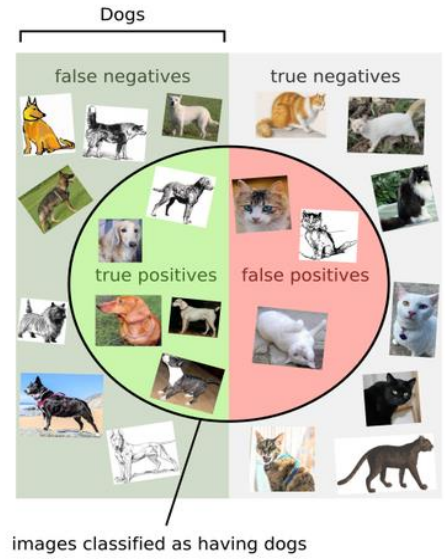
$$\text{Sensitivity} = \frac{TP}{TP+FN}$$



False positive rate

- **False positive rate or false alarm rate**
- the incorrectly classified negative samples to the total number of negative samples
- FPR = 1 – specificity (TNR)

$$FPR = 1 - TNR = \frac{FP}{FP + TN} = \frac{FP}{N}$$



F1 score

- The *F*1 score, *F* score, or *F* measure is the harmonic mean of precision and sensitivity it gives importance to both factors

$$F1 = \frac{2(Precision)(Recall)}{Precision + Recall}$$

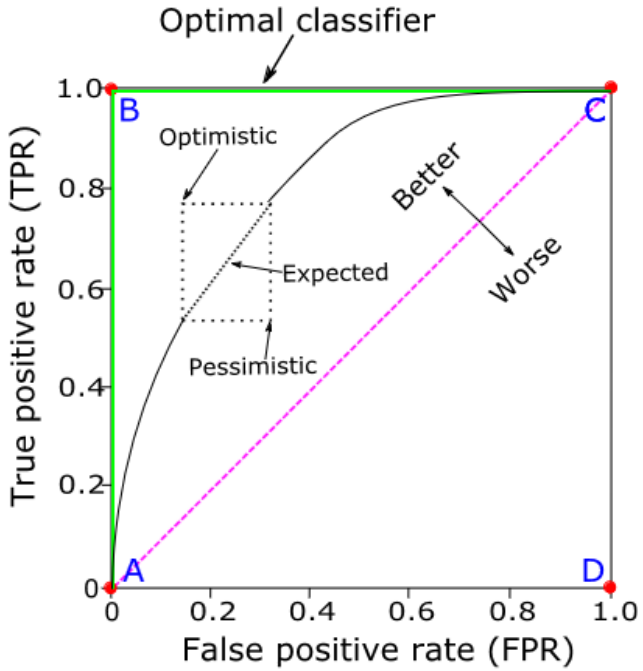
- **Macro F1** is the **average of the F1-scores calculated separately for each class**, giving **equal importance to every class**, regardless of how many samples each class contains. For K classes:

$$F1_{\text{macro}} = \frac{1}{K}\sum_{i=1}^{K} F1_i$$

where $F1_i$ is the F1-score for class i.

AUC-ROC curve

- The receiver operating characteristics (ROC) curve is the plot between sensitivity (TP rate) and the FP rate for various threshold values.
- The area under curve (AUC) is the area under this ROC curve; it is used to measure the quality of a classification model.
  - The larger the area, the better the performance.
- ROC curve is not a good visual illustration for highly imbalanced data



Thank you

Workshop Sponsors







